ImpactBridge

Replacing a WhatsApp-and-spreadsheet coordination process with scheduled automation, a predictive early-warning model, and measurable adoption

Saatvika Chokkapu

saatvikachokkapu03@gmail.com | github.com/saatvika-chokkapu/ImpactBridge | impact-bridge-saatvika.vercel.app

Work sample prepared for the AI Operations Analyst role, IEM, Austin TX | September 2026

Abstract

U&I is India's largest volunteer-driven education NGO. At its Visakhapatnam chapter, one coordinator managed 53 volunteers and 106 children almost entirely through WhatsApp messages and Google Sheets. While volunteering there I observed three recurring costs of that process: the coordinator sent five or more manual attendance confirmations every week, session sheets were completed only about 70% of the time, and children who were falling behind were noticed only once the problem was visible. ImpactBridge is the system I designed and built in response. It replaces the manual routine with four scheduled jobs (RSVP reminders, unconfirmed-volunteer alerts, a weekly ML pipeline, and a Monday at-risk digest), a 30-second mobile session logger, and a donor portal fed by demand forecasts. This paper describes the problem, the alternatives I considered and rejected, the architecture, the design decisions that made the automation reliable and observable, and how adoption is measured. It is explicit about what the deployed demo can and cannot claim: the platform runs against a realistic seeded dataset modeled on the chapter's real 2024–25 operations, not against live field data.

Keywords: workflow automation, process improvement, scheduled jobs, observability, adoption metrics, FastAPI, dbt, scikit-learn, LLM summarisation

1. Background and motivation

I volunteered with U&I's Visakhapatnam chapter for roughly nine months across 2023 and 2024, teaching, helping with fundraising, and organising events. The chapter is not short of care or effort. It is short of a system. Every Sunday session depends on a coordinator who must know, in advance, which volunteers will show up, which children each volunteer is paired with, which children need extra attention, and what teaching materials are running low. All of that knowledge lived in chat threads and spreadsheets, and all of it had to be pulled together by hand every week.

The motivation for ImpactBridge was simple: the coordinator was spending more time chasing information than acting on it. The question I set out to answer was whether a small, well-scoped automation layer could invert that, so that the information arrives on schedule and the coordinator's time goes to the children.

2. The problem, stated concretely

Before building anything I wrote down what the manual process actually cost, because an automation that does not address a measured cost is just a demo. The observations below come from my time at the chapter; the counts are approximate and I have kept them so throughout.

Manual step	Observed cost	Consequence
Confirming volunteer attendance	5+ WhatsApp messages by the coordinator every Fri/Sat	Saturday-morning scramble for cover; children without a teacher
Logging each child's session	~15 minutes per volunteer on a shared sheet; ~70% completion	Gaps in every child's history; no reliable trend
Spotting struggling children	Manual review of scattered notes, when time allowed	Problems noticed weeks late, if at all
Procuring materials	Requested after a shortage was noticed	Reactive fundraising; donors see an ask, not an outcome

Table 1. The four manual steps ImpactBridge targets, and why each mattered.

3. Alternatives considered

A custom platform is the most expensive option, so I evaluated lighter ones first. Each solved part of the problem and none solved the part that mattered most.

- **A better-structured shared spreadsheet.** Organises the data but still depends on someone opening it, updating it, and remembering to check it. It does not remove the "coordinator goes looking for information" step; it just tidies where they look.
- **A WhatsApp or SMS reminder bot.** Solves attendance confirmation cleanly, and I kept that idea (it became the Thursday RSVP job). It does nothing for the harder problem of noticing which children are quietly falling behind.
- **An off-the-shelf volunteer or donor CRM.** These are built around donor pipelines, not around a coordinator who needs child-level, week-by-week signal and a logging step that volunteers will actually complete mid-session on a phone.

The deciding insight was that the two halves reinforce each other: capture has to be fast enough that volunteers actually do it, and prediction is only possible if capture happens. That coupling is what justified a custom build, and it is why the 30-second logger and the ML pipeline were designed together rather than as separate features.

4. System overview

ImpactBridge is a full-stack application with three user-facing surfaces and a scheduled automation layer behind them.

Surface	User	What it replaces
Coordinator dashboard (7 sections: Home, Alerts, KPIs, Analysis, Volunteers, Kids, Funds, plus an Automation tab)	Chapter coordinator	Cross-referencing spreadsheets on Monday morning
Volunteer session logger (mobile-first, three taps per child)	Volunteers	The 15-minute shared sheet
Donor portal (public wishlist, live fund-drive progress)	Donors	Generic fundraising asks with no visible outcome

Table 2. User-facing surfaces.

4.1 Stack

Layer	Technology	Why
API and auth	Python, FastAPI, SQLAlchemy, JWT	Fast to build, typed request/response models, auto-generated docs the coordinator can read
Database	PostgreSQL	Relational data with clear foreign keys; free managed tiers available
Transformation	dbt (5 models, 16 tests)	Staging → intermediate features → marts; tests catch a wrong number before it reaches a dashboard
ML	scikit-learn: Random Forest + SMOTE (risk), Ridge regression (progress)	Small data, needs to be explainable; SMOTE handles the class imbalance of at-risk children
Scheduling	APScheduler in-process + GitHub Actions cron	See §5.2: two triggers so a sleeping free-tier host cannot silently skip a job
Email	SendGrid (console fallback in development)	Every send is logged to an audit table
Frontend	React, React Router, Recharts	One codebase for dashboard, logger, donor page, and one-tap RSVP
Optional LLM	Anthropic Claude (Haiku)	Rephrases the at-risk list into a short briefing; strict fallback (§6.3)
Tests	pytest (18 tests) + dbt tests	Token signing, adoption arithmetic, LLM guardrails, model contracts

Table 3. Technology choices and the reason for each.

5. The automation layer

5.1 The weekly cycle

The system is organised around one week of chapter life. Every job answers a question the coordinator used to answer by hand.

When (IST)	Job	Replaces	Output
Thursday 20:00	RSVP reminders	Manual Fri/Sat confirmations	One email per volunteer with signed one-tap Confirm / Decline links
Friday 08:00	Unconfirmed-volunteer alert	Saturday-morning scramble	Coordinator receives the list of unconfirmed volunteers and how many children each covers
Sunday 23:00	ML pipeline	Manual review of 100+ children's notes	Features rebuilt, risk and progress predictions written, wishlist populated
Monday 07:00	At-risk digest	Coordinator hunting for who needs help	Per-chapter briefing of flagged children with the reason for each flag
On demand	Donor impact card	Nothing existed	When a funded item is used in a session, the donor is told which child used it and when

Table 4. The five automated jobs.

5.2 Making the schedule reliable

The first version ran APScheduler inside the FastAPI process. That is fine on an always-on server and quietly wrong on the free hosting tiers an NGO would actually use, which sleep idle services. A sleeping process cannot fire Thursday's reminders, and nobody would know until Sunday.

The fix was to fire every job two ways. The in-process scheduler remains, and a GitHub Actions workflow with four cron entries wakes the API and calls the same trigger endpoints, authenticated with a shared key rather than a coordinator login. Each run records who triggered it (scheduler, API, or GitHub Actions), so the dashboard shows which path actually fired. GitHub Actions cron is free, independent of the host, and visible in the repository, which also makes the schedule reviewable by anyone reading the code.

5.3 Closing the RSVP loop safely

A reminder email is only useful if answering it takes one tap and requires no login. The link therefore carries an HMAC token bound to the specific session and volunteer. The public endpoint verifies the token in constant time, rejects sessions that have already happened, and is idempotent, so tapping twice simply re-saves the same answer. A forwarded link cannot be used to confirm on someone else's behalf. The token functions are pure and covered by unit tests.

5.4 An audit trail that actually exists

An honest note: the original email service was written to log every send to an audit table, but the table's model was never defined and the write was wrapped in a bare exception handler, so the log silently did nothing. I found this while reviewing the code for this submission. I defined the model, added a second table that records each job execution (start, finish, outcome, one-line summary, error text), and built the Automation tab on top of both so the gap cannot hide again. I mention it because catching that kind of silent failure is a large part of what operating automation, as opposed to building it, actually involves.

6. Prediction and the Monday briefing

6.1 Features

Features are built from the last four weeks of session logs per child: attendance rate, average rating, consecutive struggling sessions, whether the child is stuck at a level, and rate of chapter progression. The same logic exists as a dbt intermediate model so the numbers on the dashboard and the numbers the model sees are the same numbers.

6.2 Models

- **At-risk classifier.** Random Forest with SMOTE oversampling, because at-risk children are a small minority of the roster. Output is a risk level plus a plain-English reason (for example "2 consecutive struggling sessions"), because a coordinator will not act on a probability.
- **Progression predictor.** Ridge regression projecting each child's literacy and numeracy level four weeks out. Its purpose is not the prediction itself but the procurement it enables: a child projected to move from letters to words will need a different kit, and the wishlist is populated ahead of that need rather than after.

6.3 Optional LLM briefing, with guardrails

If an Anthropic API key is configured, the Monday digest opens with a four-to-seven-sentence briefing generated by Claude Haiku. The prompt is deliberately narrow: group children by shared reason, mention every child by name, use only the facts supplied, give no advice or diagnosis, and end with who to check on first. The output is then validated in code. If the key is missing, the request fails, the response is too short, or any child's name is absent from the text, the email falls back to the plain list. The email always goes out. The LLM improves readability; it is never allowed to be the reason a coordinator does not get the list.

7. Measuring whether it works

An automation that runs is not the same as an automation that helps. The Automation tab in the dashboard was built to answer three questions an operations reviewer would ask, in order.

Question	Where it is answered	Source of truth
Did each job run, and did it succeed?	Job health table: last run, trigger path, outcome, summary, 30-day success rate, "Run now"	automation_runs table
Is anyone using it, and did it help?	Adoption cards: session-sheet completion versus the 70% baseline, RSVP response rate, estimated volunteer hours saved, coordinator messages avoided	RSVP, session-log, and audit tables
What exactly went out, and to whom?	Audit log of every automated email with status	automation_logs table

Table 5. The three questions the Automation tab answers.

Two design rules for the adoption metrics. First, compare like with like: the 70% baseline referred to whether a volunteer who attended filled in their sheet, so "sheet completion" is measured as, of volunteers who confirmed attendance, how many logged at least one session. An earlier version counted logs per enrolled child, which penalised absences and reported 38%; that was a measurement error, not a product failure, and it is the kind of thing this panel is meant to surface. Second, label estimates as estimates: "volunteer time saved" multiplies logged sessions by the difference between the old 15-minute sheet and the 30-second logger, and the card says so.

8. Results, and what they do and do not show

The deployed system (frontend on Vercel, API and PostgreSQL on Render, schedule on GitHub Actions) is seeded with a dataset modeled on the chapter's real 2024–25 shape: 3 centres, 106 children, 53 volunteers, 44 weeks of Sunday sessions with a 30% no-show rate and 70% sheet completion built in, and an upcoming Sunday with all RSVPs pending so the reminder and alert jobs have real work to do. Against that dataset, on the day of writing:

- All four scheduled jobs run green from the dashboard and from the external cron; 53 reminders sent, 53 unconfirmed volunteers surfaced to 6 coordinators, 106 children scored, 8 flagged at risk, 6 digests sent.
- The one-tap RSVP path works end to end without a login, and rejects tampered or forwarded links.

- The eight-week adoption window shows 24 sessions, 424 RSVPs answered, 296 confirmed ahead of Sunday, and an estimated 78 volunteer-hours saved on logging.

What this does not show, and I want to be precise about it: these are results against seeded data, not field outcomes. The chapter has not adopted ImpactBridge officially. One former teammate, now a chapter leader, uses it informally. The model's headline metrics (for example an AUC-ROC of 0.97) are measured on the seeded data and should be read as "the pipeline works and is well-calibrated to the shape of the problem", not as a validated field result. The honest claim is that the process is fully automated and observable, and that the platform is ready to be measured against real usage the day a chapter turns it on.

9. Limitations and open work

- Field validation. The next real step is a pilot at one centre with the actual coordinator, measuring sheet completion and RSVP response against the manual baseline over eight weeks.
- Hygiene tracking. The dashboard's hygiene charts are sample data and are labelled as such in the UI; the data model for monthly hygiene checks does not exist yet.
- Notification channel. Volunteers in this context live on WhatsApp, not email. The email layer is a stand-in for a WhatsApp Business API integration, which is the single change most likely to move RSVP response rates in the field.
- Training and enablement. I have written documentation for the system but have not yet run a training session for a chapter team; that is a gap in the adoption story, not just in the software.

10. What I would tell the next person

Three things carried over from this project into how I approach automation generally. Write down the manual cost before building, because it becomes the only defensible yardstick later. Build the observability at the same time as the automation, not after, because the silent failures are the ones that cost trust. And measure adoption in the terms the people affected already use, or the number will be right and still mean nothing to them.

Appendix A. Reproducing and reviewing

Item	Location
Source code	github.com/saatvika-chokkapu/ImpactBridge
Live application	impact-bridge-saatvika.vercel.app (coordinator: coord_0_0@impactbridge.org / coord123; volunteer: vol_0@impactbridge.org / vol123)
API and interactive docs	impactbridge-22jw.onrender.com/docs (free tier; first request after idle can take ~30 s)
External schedule	.github/workflows/automation.yml (four cron entries, manual dispatch supported)
Automation tab	Dashboard → ⚙ Automation: job health, adoption metrics, email audit log, "Run now" for each job
Tests	pytest -q tests (no database required); dbt test in impactbridge_dbt/

Item	Location
Design notes	README.md: problem, alternatives, flowchart, and a “what changed along the way” section

Table A1. Everything a reviewer needs to verify the claims above.

All work in this document was scoped and built by me. AI coding assistants (Claude Code, Cursor) were used in development, in the same way I use them daily; design decisions, the problem framing, and the evaluation approach are my own.